UNIVERSIDADE FEDERAL DE ALFENAS

GUSTAVO BORIN NASCIMENTO

AVALIAÇÃO DE RANGE PROOFS EM NOIR/ULTRAHONK PARA VALIDAÇÃO PRIVADA DE ATRIBUTOS COMPROMETIDOS EM CONTRATOS INTELIGENTES COMPATÍVEIS COM ETHEREUM

ALFENAS – MG

2026

GUSTAVO BORIN NASCIMENTO

AVALIAÇÃO DE RANGE PROOFS EM NOIR/ULTRAHONK PARA VALIDAÇÃO PRIVADA DE ATRIBUTOS COMPROMETIDOS EM CONTRATOS INTELIGENTES COMPATÍVEIS COM ETHEREUM

Trabalho de Conclusão de Curso em Ciência da Computação da Universidade Federal de Alfenas. Monografia em preparação.
Orientador: Iago Augusto de Carvalho

ALFENAS – MG
2026

Ficha catalográfica: pendente de emissão, quando aplicável.

RESUMO

Rascunho para orientação, atualizado em 11 de novembro de 2026. A implementação, os resultados funcionais, estruturais e de gas estão documentados; dez dos vinte pares diários foram concluídos. A análise final de latência, a conclusão e este resumo permanecem pendentes. Este documento não é a versão de entrega.

Palavras-chave: A definir após a redação do resumo.

ABSTRACT

Draft for advisor review, updated on November 11, 2026. Ten of twenty daily pairs are complete. The final latency analysis, conclusions and abstract remain pending. This is not the submission version.

Keywords: To be defined after the abstract is written.

LISTA DE TABELAS

Tabela 1 – Operações e tamanho das provas das oito variantes 27
Tabela 2 – Tamanho dos modelos compilados de código dos verificadores 28
Tabela 3 – Gas de verificação das oito variantes no fork local de Sepolia 28
Tabela 4 – Gas total de implantação dos verificadores e bibliotecas vinculadas 29

SUMÁRIO

1 Introdução 8
1.1 CONTEXTO E PROBLEMA DE PESQUISA 8
1.2 JUSTIFICATIVA E PERGUNTAS DE PESQUISA 9
1.3 OBJETIVOS 9
1.4 DELIMITAÇÃO E ORGANIZAÇÃO DO TRABALHO 10
2 Fundamentação e trabalhos relacionados 11
2.1 PROVAS DE CONHECIMENTO ZERO E PROVAS DE INTERVALO 11
2.2 COMPROMISSOS E RELAÇÕES VERIFICADAS 11
2.3 NOIR, ACIR, ULTRAHONK E AMBIENTES DE EXECUÇÃO 12
2.4 VERIFICAÇÃO EVM E COMPONENTES DE CUSTO 13
2.5 TRABALHOS RELACIONADOS E POSICIONAMENTO 14
3 Implementação 17
3.1 RELAÇÕES E ENTRADAS DAS OITO VARIANTES 17
3.2 ARQUITETURA E GERAÇÃO DAS PROVAS 18
3.3 TESTES POSITIVOS E NEGATIVOS 19
4 Método experimental 20
4.1 DESENHO, AMBIENTE E UNIDADE EXPERIMENTAL 20
4.2 COLETA DIÁRIA E TRATAMENTO DE FALHAS 21
4.3 MÉTRICAS ESTRUTURAIS E GAS 22
4.4 ANÁLISE ESTATÍSTICA E JUSTIFICATIVA DA REVISÃO 23
4.5 LIMITAÇÕES DO DESENHO 25
5 Resultados 26
5.1 COBERTURA E CORREÇÃO FUNCIONAL 26
5.2 CUSTOS ESTRUTURAIS E VERIFICAÇÃO EVM 26
5.3 EXECUÇÃO DA COLETA E INCIDENTES 29
5.4 LATÊNCIA E CONTRASTES DIÁRIOS 30
6 Discussão 31
6.1 RESPOSTAS ÀS PERGUNTAS DE PESQUISA 31
6.2 COMPARAÇÃO COM TRABALHOS RELACIONADOS 32
6.3 IMPLICAÇÕES E AMEAÇAS À VALIDADE 32
7 Conclusão 34

7.1 SÍNTESE DAS RESPOSTAS E CONTRIBUIÇÕES 34
7.2 LIMITAÇÕES E TRABALHOS FUTUROS 34
REFERÊNCIAS 35
APENDICE A – RASTREABILIDADE DOS ARTEFATOS E DOS RESULTADOS 38

1 INTRODUÇÃO

1.1 CONTEXTO E PROBLEMA DE PESQUISA

Verificar uma condição sobre um valor não exige necessariamente conhecer esse valor. Uma aplicação pode precisar verificar se um atributo pertence a um intervalo, enquanto seu titular deseja preservar o atributo como informação privada. Provas de conhecimento zero oferecem uma formulação criptográfica para verificar afirmações sem revelar informação adicional sobre a testemunha, segundo as propriedades e o modelo do sistema utilizado (1). No caso de uma prova de intervalo, a condição de interesse é $a \leq v \leq b$, com limites públicos a e b e valor privado v.

Para relacionar essa condição a um atributo previamente comprometido, o circuito precisa vincular o mesmo valor v a um compromisso público. Esse vínculo integra o objeto deste trabalho: além das relações de intervalo, uma variante verifica a consistência entre o valor privado e o compromisso informado. O compromisso delimita o objeto sobre o qual se faz a prova, mas não estabelece, por si só, quem forneceu o atributo nem se ele corresponde a uma informação verdadeira sobre uma pessoa ou instituição.

A implementação dessa relação envolve custos em etapas diferentes. O provador produz um artefato que será consumido pelo verificador; o programa de alto nível, sua representação intermediária e o sistema de provas participam dessa cadeia. A literatura de benchmarking de provas ressalta a necessidade de distinguir essas camadas e considerar múltiplas métricas (2). Assim, a contagem de operações do circuito, o tempo de geração, o tamanho da prova e o custo de verificação respondem a perguntas complementares.

Este trabalho estuda a cadeia Noir/Barretenberg com UltraHonk. Compara a geração de provas por um processo nativo e por WebAssembly em navegador, além de diferentes representações numéricas e da adição do compromisso. A verificação utiliza contratos Solidity em um ambiente EVM local. A questão central é: quais custos computacionais e de verificação essa configuração apresenta para as relações implementadas, e como esses custos variam entre os modos de execução e as representações selecionadas?

1.2 JUSTIFICATIVA E PERGUNTAS DE PESQUISA

A avaliação conjunta dessas etapas é relevante para decisões de implementação. A escolha de executar o provador no navegador envolve um ambiente operacional distinto do processo nativo; já a verificação em contrato introduz custos de transação e restrições de tamanho do código. A EIP-170, por exemplo, estabelece um limite para o código implantado no regime em que se aplica (3). Portanto, a geração bem-sucedida de uma prova não basta para caracterizar todos os custos do fluxo.

No eixo das representações, comparar tipos numéricos exige controlar o significado da relação. Uma variante sobre corpo finito que não verifica o intervalo não constitui uma alternativa semanticamente equivalente a um circuito que o verifica. O estudo separa esse caso diagnóstico das comparações entre inteiros e representações sobre corpo finito com restrições explícitas. Essa distinção permite interpretar diferenças de custo sem atribuí-las apenas ao nome do tipo utilizado.

A questão central é desdobrada em três perguntas:

P1: Como diferem as latências das oito variantes entre o processo nativo e o navegador, nas condições da máquina e do protocolo adotados?

P2: Que diferenças de latência aparecem entre representações semanticamente comparáveis e com a inclusão da relação de compromisso?

P3: Quais são os custos estruturais, os tamanhos dos artefatos e os custos de implantação e verificação EVM das oito variantes?

Essas perguntas operacionalizam a avaliação de viabilidade do projeto em termos de métricas observáveis. A interpretação de adequação a uma aplicação específica dependeria ainda de seus requisitos de latência, recursos e custo, não definidos por este experimento.

1.3 OBJETIVOS

O objetivo geral é desenvolver, implantar e avaliar circuitos de provas de intervalo em Noir/UltraHonk para validação privada de atributos comprometidos em contratos inteligentes compatíveis com Ethereum, dentro do recorte experimental definido.

Os objetivos específicos são implementar as relações de intervalo e a variante vinculada a compromisso; verificar o comportamento dos circuitos com entradas válidas e inválidas;

compilar os circuitos e gerar seus verificadores Solidity; comparar a latência de geração nos ambientes nativo e WebAssembly; comparar representações inteiras com variantes sobre corpo finito semanticamente controladas; e caracterizar operações ACIR, gates do backend, tamanhos de prova e de código, gas de implantação e de verificação e custos de calldata. A análise distingue o gas total do recibo de componentes cuja decomposição seja sustentada pelas regras efetivamente executadas.

1.4 DELIMITAÇÃO E ORGANIZAÇÃO DO TRABALHO

O estudo abrange oito variantes: três circuitos inteiros, três circuitos sobre corpo finito com restrições explícitas correspondentes, um circuito diagnóstico sobre corpo finito e um circuito de intervalo com compromisso. As medições de latência utilizam uma máquina, versões fixadas e entradas canônicas. A comparação entre ambientes considera seus escopos operacionais de medição, que diferem no carregamento e na permanência do runtime. As diferenças observadas não isolam o efeito causal do motor de execução.

A avaliação se concentra na implementação das relações e em seus custos. Não inclui uma infraestrutura de emissão de credenciais, autenticação de atributos ou uma aplicação completa de identidade ou finanças. A verificação EVM local não representa uma avaliação de congestionamento de rede ou de preços monetários. Os testes negativos fornecem evidência sobre os casos exercitados, sem substituir uma demonstração formal de segurança da cadeia criptográfica.

A contribuição pretendida é uma avaliação rastreável dessa configuração, acompanhada da implementação, dos artefatos e de um procedimento de coleta e análise reproduzível. O método distingue pilotos de coleta final e explicita as condições para a inferência. Nesta versão, a caracterização estrutural e EVM de P3 está concluída no recorte adotado; as respostas de latência de P1 e P2 dependem do encerramento e da validação da coleta diária.

A Seção 2 apresenta os conceitos e trabalhos relacionados. A Seção 3 descreve as relações e o fluxo implementado, e a Seção 4 estabelece o método experimental. Os resultados são apresentados na Seção 5, seguida da discussão na Seção 6 e da conclusão na Seção 7.

2 FUNDAMENTAÇÃO E TRABALHOS RELACIONADOS

A avaliação proposta envolve três níveis: a relação que se deseja verificar, sua representação em circuitos e o custo de executar os componentes do sistema. Separar esses níveis permite interpretar por que uma mudança de tipo pode modificar a implementação sem mudar a relação, enquanto a adição de um compromisso modifica a própria afirmação verificada.

2.1 PROVAS DE CONHECIMENTO ZERO E PROVAS DE INTERVALO

Uma relação $R(x, w)$ associa uma instância pública x a uma testemunha w. O provador utiliza a testemunha para sustentar a afirmação, enquanto o verificador avalia sua aceitação. A referência da comunidade ZKProof distingue completude, correção contra provadores desonestos e conhecimento zero. A completude trata da aceitação quando a afirmação e a execução honesta são válidas; a correção limita a aceitação de afirmações falsas. Conhecimento zero é definido por simulação e limita a informação adicional revelada sobre a testemunha. Essas propriedades possuem definições formais e dependem do modelo adotado (1).

Neste trabalho, a instância pública inclui os limites do intervalo, e a testemunha inclui o valor privado. Uma prova de intervalo estabelece que esse valor pertence a um intervalo especificado. A proposta de benchmarking da comunidade ZKProof inclui essa funcionalidade e distingue a relação de alto nível da representação utilizada para implementá-la (2). A distinção é relevante porque representar um valor em um corpo finito não impõe, por si só, a interpretação inteira pretendida pelo intervalo.

A expressão “prova” será utilizada para os artefatos gerados pelo sistema estudado. A aceitação dos testes de implementação não será apresentada como demonstração das propriedades formais do protocolo. Da mesma forma, manter um valor como entrada privada não torna privados os limites, identificadores ou demais dados explicitamente públicos.

2.2 COMPROMISSOS E RELAÇÕES VERIFICADAS

Para o protótipo, um compromisso público permite formular uma relação que liga o valor privado a um objeto já conhecido pelo verificador. A afirmação deixa de ser apenas “existe um valor no intervalo” e passa a exigir que o valor usado na comparação também participe do cálculo do compromisso fornecido. Essa ligação é expressa no próprio circuito, conforme a Equação 3.3.

A implementação utiliza uma função Pedersen da biblioteca adotada, um valor auxiliar privado e um identificador público. A formulação é específica do protótipo e não deve ser confundida automaticamente com qualquer construção denominada compromisso de Pedersen na literatura. Sua finalidade experimental é medir a relação adicional implementada. Autenticação de emissor, origem do atributo e proteção contra reutilização exigiriam mecanismos além dessa igualdade.

As provas de intervalo sobre valores comprometidos são um tema da literatura criptográfica. Bulletproofs apresenta uma construção não interativa sem configuração inicial confiável, com tamanho de prova logarítmico na dimensão considerada, e trata explicitamente de intervalos sobre valores comprometidos (4). Essa construção fornece contexto para a funcionalidade, mas não é o protocolo executado neste estudo.

2.3 NOIR, ACIR, ULTRAHONK E AMBIENTES DE EXECUÇÃO

Noir é uma linguagem específica de domínio voltada à construção de programas que utilizam provas de conhecimento zero (5). No fluxo deste projeto, o programa é compilado para ACIR e posteriormente processado pelo backend Barretenberg para geração das provas UltraHonk. A cadeia efetivamente utilizada, seus artefatos e versões são apresentados na Seção 3.

A documentação pública do Noir consultada identifica uma versão posterior à instalada neste experimento. Ela é utilizada aqui apenas para caracterizar a finalidade da linguagem. As afirmações sobre APIs, relações e configuração executada dependem dos artefatos e fontes da versão congelada. Não se presume que uma API atual descreva retroativamente o comportamento da versão experimental.

Uma representação intermediária e um backend cumprem funções distintas. Benarroch et al. descrevem a transformação de programas em representações verificáveis e o processamento posterior pelo sistema de provas, ressaltando que os custos dependem das duas camadas (2). Essa separação orienta a apresentação conjunta de operações ACIR e gates: as duas contagens não são unidades intercambiáveis, e nenhuma delas determina isoladamente a duração observada.

Na documentação associada à versão adotada do Barretenberg, Honk combina restrições polinomiais do circuito, o protocolo Sumcheck e compromissos polinomiais KZG. Sumcheck reduz uma afirmação sobre uma soma de avaliações a verificações de avaliações polinomiais.

A etapa de compromissos emprega Gemini para transformar afirmações multilineares em univariadas e Shplonk para agrupá-las antes da abertura KZG. A configuração Ultra inclui relações aritméticas, de permutação, de consulta a tabelas e de intervalo, entre outras (6).

Essa arquitetura distingue a relação escrita pelo programador das operações executadas pelo provador. O transcript serializa mensagens e deriva desafios pela transformação Fiat–Shamir; as configurações destinadas à verificação EVM utilizam Keccak. As variantes com conhecimento zero incorporam mecanismos de mascaramento (6). No experimento, o alvo EVM é mantido nos dois ambientes, conforme o método, para preservar a configuração criptográfica da comparação.

WebAssembly é um formato de código de baixo nível projetado para portabilidade e execução eficiente, utilizável em browsers e em outros ambientes. A especificação distingue o núcleo da linguagem das interfaces fornecidas pelo ambiente que a hospeda (7). Seus objetivos de desempenho não constituem garantia de igualdade com uma execução nativa para qualquer aplicação. Neste estudo, a comparação envolve um processo de linha de comando e um runtime residente no browser, com diferenças de carregamento e instrumentação descritas no método.

2.4 VERIFICAÇÃO EVM E COMPONENTES DE CUSTO

A utilização de verificadores Solidity introduz restrições distintas do custo de geração da prova. A EIP-170 limita a 24.576 bytes o código retornado pela inicialização do contrato, no regime em que essa regra se aplica (3). Portanto, tamanho do código implantado e gas de verificação são medidas complementares: atender ao limite de código não demonstra baixo custo de execução.

A composição da calldata também interfere no custo de uma transação. A EIP-7623 define um piso dependente dos bytes de dados, combinando o componente convencional com um mínimo por tokens de calldata (8). Quando esse piso determina o total, o recibo não identifica univocamente o componente de execução por simples subtração. A aplicação da regra ao experimento depende da confirmação do hardfork efetivamente executado, não apenas de consultar a fórmula.

A EIP-4844 introduz transações com blobs e uma contabilização específica. Os dados completos dos blobs não ficam diretamente acessíveis à execução EVM como calldata; a especificação utiliza compromissos e interfaces próprias (9). Por isso, projetar ocupação de bytes

em blobs não equivale a implementar uma substituição direta dos dados consumidos pelo verificador nem demonstra economia monetária.

2.5 TRABALHOS RELACIONADOS E POSICIONAMENTO

Os trabalhos selecionados cumprem funções diferentes nesta pesquisa. Bulletproofs trata da construção criptográfica e de provas de intervalo sobre compromissos, incluindo agregação (4). O presente estudo utiliza circuitos Noir e UltraHonk, sem implementar Bulletproofs como concorrente. Consequentemente, seus resultados publicados não serão colocados ao lado das medições locais como se constituíssem uma comparação controlada de desempenho.

A proposta de Benarroch et al. organiza desafios de comparação de sistemas de provas e múltiplas métricas de custo (2). O recorte deste TCC é mais restrito: manter a cadeia de ferramentas e comparar relações e modos de execução específicos. A contribuição pretendida é uma avaliação rastreável dessa configuração, e não um ranking geral de sistemas criptográficos. A proposta comunitária é uma referência metodológica, não uma certificação do procedimento local.

Kalibera e Jones estudam fontes de variação em benchmarking de sistemas e a distribuição de repetições entre níveis como construções, execuções e iterações (10). Essa perspectiva fundamenta a necessidade de distinguir ciclos internos de sessões reiniciadas. Entretanto, o artigo não prescreve os 20 dias deste experimento nem valida automaticamente o estimador de mediana diária escolhido. A alocação e as premissas específicas são justificadas na Seção 4.

No eixo de abrangência da avaliação, zk-Bench propõe examinar diferentes camadas da implementação, desde bibliotecas aritméticas até ferramentas de desenvolvimento de SNARKs, e inclui configurações distintas de hardware (11). Esse escopo contrasta com o desenho deste TCC, que fixa uma máquina e uma cadeia de ferramentas. A comparação local concentra-se nas relações implementadas e nos ambientes de execução; a cobertura de oito variantes não representa diversidade de plataformas criptográficas.

No eixo browser/nativo, Jangda et al. avaliam aplicações da suíte SPEC CPU com Browsix-Wasm e encontram diferenças de desempenho entre código nativo e WebAssembly executado em browsers (12). O trabalho oferece um antecedente para a avaliação empírica desse eixo, mas suas cargas não são os provadores deste estudo. Versões dos motores, aplicações e

configuração de execução delimitam os resultados publicados, impedindo que seus fatores de desempenho sejam usados como previsão para UltraHonk.

Há também avaliações diretamente voltadas a provadores Noir. O repositório ZKE-mail.nr Benchmarks distingue navegador/WASM, Node.js e execução nativa para um circuito de verificação DKIM, apresentando separadamente primeira execução, agregados posteriores e geração de testemunha (13). Essa separação evita identificar toda execução JavaScript como execução em navegador. Contudo, no código web consultado, o acumulador exclui a primeira execução e soma uma execução a menos que o divisor usado para a média. A correspondência desse código com os valores publicados não está demonstrada; os agregados não são utilizados aqui para estimar fatores de desempenho.

Duff compara provas de pertencimento e não pertencimento em árvores Merkle nos modos nativo e WASM, declarando entradas e artefatos compilados idênticos. A metodologia separa geração de testemunha, geração de prova e inicialização, excluindo esta última das tabelas de dez execuções (14). Os trechos metodológicos e tabelas consultados não identificam suficientemente hardware, navegador e threads para reconstruir o ambiente da comparação. A contribuição para este TCC é a explicitação de fases e controles declarados; a interpretação local permanece vinculada às fronteiras de medição e à configuração efetivamente registradas neste experimento.

No eixo geração/verificação, o relatório técnico de Nadolinski para a Base compara SnarkJS, Rapidsnark, Gnark e Noir/Barretenberg na prova de verificação de assinaturas ECDSA P-256, medindo tempo de geração e gas em instâncias AWS EC2 (15). O relatório identifica a execução de Noir usada nos benchmarks como nativa, embora mencione suporte a browser. A proximidade com este TCC está nas métricas e na presença de UltraHonk; a relação criptográfica e o hardware são diferentes. Por isso, o relatório contextualiza os componentes de custo, sem fornecer uma comparação controlada com nossas provas de intervalo.

No plano da documentação experimental, a campanha V1 do ProveKit define uma fronteira entre entrada estruturada e prova serializada, distingue runtime novo de runtime reutilizado e mantém indisponibilidades separadas de valores medidos (16). A superfície publicada utiliza Chrome/WASM em um Mac e execução nativa em dispositivos móveis; portanto, esses alvos não isolam o efeito do ambiente no mesmo hardware. O documento também explicita diferenças entre as afirmações provadas pelos sistemas comparados. Sua utilidade para este estudo

está na rastreabilidade das condições e na delimitação das comparações, não na transferência de resultados numéricos.

No plano da aplicação, Gasser, Humbert e Elghareeb apresentam uma prova de conceito de credenciais para identidade eletrônica que inclui um predicado de idade em Noir (17). O relatório distingue esse predicado dos mecanismos de vínculo com o titular e de verificação da assinatura do emissor, indicando que precisam ser combinados para sustentar a apresentação de uma credencial válida. Essa distinção ajuda a delimitar o protótipo deste TCC: verificar um intervalo e sua consistência com um compromisso não implementa essas demais garantias. Os custos do predicado isolado não são adotados como custos de uma credencial completa.

A seleção reúne três eixos complementares: camadas do sistema, ambiente de execução e custo de geração/verificação. O recorte deste trabalho combina esses eixos em uma configuração delimitada, com relações explicitamente descritas e medições rastreáveis. A busca realizada é dirigida, não sistemática; a seleção não sustenta alegação de exclusividade ou ausência de estudos semelhantes.

3 IMPLEMENTAÇÃO

A implementação reúne oito circuitos para examinar custos de representação numérica, ambiente de execução e vinculação de um valor a um compromisso. Cada circuito produz uma relação verificável com entradas públicas e privadas explicitadas. A finalidade dessa organização é permitir comparações delimitadas: variantes que verificam relações distintas não são tratadas como soluções semanticamente equivalentes.

3.1 RELAÇÕES E ENTRADAS DAS OITO VARIANTES

Nas variantes de intervalo, o valor v é privado e os limites a e b são públicos. Para $N \in \{8, 32, 64\}$, a relação de interesse é

$$0 \leq a \leq v \leq b < 2^N. \tag{3.1}$$

As três variantes com tipos inteiros sem sinal, denominadas `u8`, `u32` e `u64`, utilizam o tipo correspondente e verificam as duas comparações entre o valor e os limites. Os extremos do intervalo são admitidos.

Outras três variantes representam os valores como elementos de corpo e impõem restrições explícitas de tamanho em bits. Elas são denominadas Field com restrições de 8, 32 e 64 bits. Cada uma limita v, a, b, $v-a$ e $b-v$ a N bits. Restringir apenas as entradas não seria suficiente para justificar as duas desigualdades a partir das diferenças calculadas no corpo.

Seja p o módulo do corpo e $M = 2^N - 1$. Depois de restringir as entradas ao intervalo $[0, M]$, cada diferença inteira pertence a $[-M, M]$. Uma diferença negativa, representada módulo p, pertence a $[p-M, p-1]$. Para as larguras utilizadas, $p > 2M$, de modo que esse conjunto não intersecta $[0, M]$. Portanto, exigir que as diferenças também caibam em N bits exclui esses representantes de diferenças negativas. Esse argumento explica a relação implementada; não constitui uma auditoria formal do compilador ou do sistema de provas.

A sétima variante, denominada Field diagnóstico, verifica somente

$$(v-a)(b-v) \neq 0 \tag{3.2}$$

no corpo. Essa condição exclui $v = a$ e $v = b$, mas admite valores fora do intervalo inteiro. Por isso, ela é mantida como carga aritmética de diagnóstico, e não como prova de pertencimento ao intervalo da Equação 3.1. Sua presença não autoriza atribuir diferenças de custo exclusivamente à representação numérica.

A oitava variante acrescenta um compromisso à relação de intervalo de 64 bits. O circuito recebe v e um valor auxiliar s como entradas privadas; os limites, um identificador id e o compromisso C são públicos. Além das desigualdades, verifica

$$C = \text{PedersenHash}(d, v, s, id), \qquad d = \texttt{0x54434301}. \tag{3.3}$$

A ordem dos quatro elementos é fixa e o mesmo v participa da comparação de intervalo e do cálculo do compromisso. O identificador é um elemento de corpo; uma aplicação que utilize identificadores textuais necessitaria definir sua codificação.

O circuito comprova consistência com o compromisso fornecido. Ele não autentica a entidade emissora nem estabelece a veracidade de um atributo no mundo real. O valor auxiliar divulgado nos vetores de teste serve à reprodução e não oferece segredo em uma aplicação. A segurança de um uso real requer análise própria, incluindo geração e manutenção desse segredo, tratamento de identificadores e possibilidade de reutilização das provas.

3.2 ARQUITETURA E GERAÇÃO DAS PROVAS

O fluxo de implementação separa compilação do circuito, preparação da testemunha, geração da prova e verificação. A versão utilizada do compilador Noir é `1.0.0-beta.20`; Barretenberg e sua interface JavaScript utilizam `5.0.0-nightly.20260324`. Os artefatos avaliados usam o alvo de verificação EVM, preservando a configuração necessária aos verificadores Solidity empregados no estudo.

As entradas canônicas dos ensaios são $v = 50$, $a = 10$ e $b = 100$. Na variante com compromisso, o identificador canônico é 7, com valor auxiliar e compromisso registrados no conjunto de validação. Esses valores fixos tornam as entradas reproduzíveis, mas não representam uma amostra de atributos de uma aplicação.

No ambiente nativo, cada tentativa invoca o processo de geração com circuito, testemunha e chave de verificação previamente preparados. No browser, a implementação utiliza explicitamente o backend `WasmWorker`, com entradas carregadas e descomprimidas antes da região cronometrada. A prova produzida é persistida e verificada contra a chave correspondente, incluindo a comparação das entradas públicas com os valores esperados.

A proveniência retém fontes, artefatos compilados, testemunhas, chaves, configurações e resumos criptográficos dos arquivos. Essa identificação permite detectar divergências entre o

material validado e o material usado na coleta. O registro de hashes comprova identidade dos bytes conferidos; a validade criptográfica é verificada por uma operação separada.

3.3 TESTES POSITIVOS E NEGATIVOS

A validação funcional precede a avaliação de desempenho. A suíte Noir compreende 71 funções de teste, incluindo valores internos, extremos, intervalos degenerados e rejeição de limites invertidos. As variantes Field com restrições também são exercitadas com valores fora do domínio e representantes associados ao retorno modular. Na variante com compromisso, os testes alteram valor, segredo auxiliar, identificador, compromisso, domínio e ordem dos elementos.

O conjunto arquivado de validação externa registra 46 casos inválidos rejeitados e provas canônicas válidas para as oito variantes. A suíte Foundry registra 15 funções de teste, com verificações de aceitação, corrupção de prova, alteração de entradas públicas, quantidade incorreta de entradas e incompatibilidade de chaves. O número de funções não deve ser confundido com o número de combinações exercitadas, pois algumas funções percorrem vários verificadores e casos.

Esses testes oferecem evidência de correção para os casos executados e proteção contra regressões conhecidas. Não constituem demonstração exaustiva de segurança criptográfica. A comparação do vetor de compromisso com a interface JavaScript verifica consistência entre interfaces da mesma família de backend, não concordância entre implementações criptográficas independentes.

4 MÉTODO EXPERIMENTAL

Esta seção descreve o protocolo de coleta e análise. Em 11 de novembro de 2026, a rodada de gas das oito variantes e dez dos 20 pares diários estavam concluídos. A meta de vinte pares foi mantida; os dez restantes e a análise final de latência permanecem pendentes. Os resultados executados são apresentados na Seção 5.

4.1 DESENHO, AMBIENTE E UNIDADE EXPERIMENTAL

O estudo avalia as oito relações da Seção 3 em uma única configuração de hardware e software. O ambiente congelado registra Linux com kernel 6.17.0-40-generic, 12 processadores lógicos disponíveis na afinidade e política de frequência identificada como `powersave`. Esses registros não implicam frequência constante. O browser é Chrome 150.0.7871.186, com V8 15.0.245.21; o backend WASM informou 12 threads na verificação de prontidão. O tamanho do conjunto de trabalhadores nativos não foi observado pelo coletor.

Os metadados de prontidão identificam o processador Intel Core i5-13420H e registram `MemTotal` de 24.304.624 KiB, aproximadamente 23,18 GiB, disponibilizados ao sistema operacional. Esse registro é distinto da memória livre ou disponível durante cada sessão e não é uma medição de consumo do provador.

A unidade de replicação da análise primária é o par diário, formado por uma sessão nativa e uma sessão WASM. Os ciclos de uma mesma sessão não são contados como replicações independentes. A inferência pressupõe uma aproximação de independência e identidade de distribuição dos resumos diários; espaçar as sessões entre dias não demonstra essa propriedade. O alcance pretendido restringe-se ao protocolo e à máquina estudada, sem amostragem de outros equipamentos ou usuários.

As regiões cronometradas refletem modos operacionais distintos. No nativo, mede-se a execução de um novo subprocesso de geração, incluindo carregamento de arquivos. No browser, mede-se a chamada de geração no runtime residente, com dados previamente carregados e chave de verificação preparada. Conversão dos resultados, comunicação com o controlador, persistência e verificação ficam fora dessa região. Assim, a comparação não isola apenas o motor de execução.

4.2 COLETA DIÁRIA E TRATAMENTO DE FALHAS

O desenho prevê 20 dias distintos, com um par por dia. A alocação inicial distribuiu dez pares com início nativo e dez com início WASM, em ordem sorteada. O calendário passou por emendas logísticas, preservando as datas reais, as ordens e as sementes efetivamente executadas. Em 11 de novembro, havia dez pares completos; uma emenda fixou os dez restantes entre 12 e 21 de novembro, reutilizando, na ordem do calendário anterior, as entradas ainda não executadas e preservando suas sementes. As ausências permaneceram registradas, sem reduzir a amostra planejada. Cada sessão contém 15 blocos de aquecimento e 100 blocos medidos, com oito variantes por bloco em ordem aleatória definida por uma semente registrada. O total planejado para os vinte pares completos é de 40 sessões e 36.800 tentativas: 4.800 de aquecimento e 32.000 medidas.

Antes de cada sessão haverá pausa de 120 segundos. O orçamento de início de sessões do par é de 20 minutos: ultrapassado esse período, a sessão seguinte não será iniciada. Uma tentativa ativa continua sujeita ao mecanismo de timeout. O limite nominal por tentativa é de 180 segundos, aplicado ao subprocesso nativo ou ao watchdog do controlador do browser; neste último, o prazo também abrange a comunicação de início e retorno e não equivale à região de latência medida.

A primeira falha de geração, verificação ou interrupção encerra o par, preservando os registros. A regra geral não prevê reposição automática, repetição para obter um resultado completo ou substituição de latência por um limite de timeout. As exceções autorizadas durante a execução são identificadas na Seção 5, com preservação das tentativas anteriores. Os relatórios distinguirão tentativas planejadas, iniciadas, bem-sucedidas, falhas retornadas, interrompidas e não iniciadas. Dias posteriores continuam sujeitos ao calendário e aos controles de prontidão. O protocolo suspende a inferência de latência de uma condição se mais de 15% das tentativas iniciadas falharem; a exigência de 20 pares completos já impede a análise primária quando o encerramento por falha deixa um par incompleto.

O coletor confere versões e hashes antes de executar, impede concorrência entre pares e recusa sobrescrever o diretório do dia. As condições de máquina são registradas nos extremos das sessões. A visibilidade e o foco da página são observados antes e depois de cada chamada cronometrada. Não há observação contínua de carga ou temperatura durante a medição. A

ausência de dados de energia ou de algum sensor será registrada como indisponibilidade, sem pressupor alimentação ou estabilidade.

Os pilotos permanecem separados da amostra final. O relato do autor sobre uso concorrente do computador em sessões anteriores não é uma medição de carga nem demonstra a causa de diferenças observadas. Os valores de aquecimento e repetição foram mantidos como escolhas operacionais, sem afirmar convergência do aquecimento apenas pela ausência de um sinalizador de deriva.

4.3 MÉTRICAS ESTRUTURAIS E GAS

As métricas estruturais distinguem contagem de operações ACIR, contagem de gates do backend, tamanho da prova e tamanho do bytecode. Essas medidas descrevem camadas distintas e não são substitutas diretas de latência.

A rodada de gas utilizou as oito variantes em um único fork local de Sepolia, fixado no bloco 11.999.881, com o hash conferido antes das transações. Foram selecionadas cinco provas por variante nas posições absolutas 15, 40, 65, 90 e 114 da sessão nativa do primeiro par do piloto, conforme alocação anterior à rodada. Cada prova foi novamente verificada com a chave arquivada e o alvo EVM. Após implantar os verificadores e suas bibliotecas, confirmou-se o retorno verdadeiro de cada chamada de verificação e executaram-se duas transações por prova. As 80 transações permitem comparar repetições das mesmas entradas; não são tratadas como unidades independentes de uma população de provas.

O Anvil 1.1.0 executou o hardfork Prague. A compilação preservou solc 0.8.28, alvo Cancun e parâmetro `optimizer_runs=1`, distinguindo a configuração do compilador das regras do ambiente de execução. O código implantado foi comparado aos artefatos compilados, incluindo endereços de bibliotecas vinculadas e constantes inseridas pelo construtor. O endpoint remoto forneceu apenas consultas para o fork; todas as transações foram enviadas ao processo local. Foram preservados o cabeçalho do bloco, as versões, as fontes, o código implantado, as chamadas, a calldata e os recibos.

A decomposição segue a regra da EIP-7623 (8). Para z bytes nulos e n bytes não nulos na calldata, define-se $t = z + 4n$. O componente padrão custa $4t$ e o piso da transação é $21\,000 + 10t$. Quando o gas total observado excede esse piso, a execução líquida é obtida subtraindo $21\,000 + 4t$ do total; quando o piso vincula, o recibo não determina univocamente a execução.

Um diagnóstico separado, com 1.000 bytes não nulos enviados a um endereço sem código, verificou o comportamento do piso para esse vetor. Essa verificação é mais restrita que uma auditoria completa do hardfork.

O resumo por variante apresenta mínimo, mediana e máximo entre as cinco provas selecionadas, usando a primeira transação de cada par e conferindo a diferença para a segunda. A seleção por posição não constitui amostra probabilística de todas as provas possíveis. A projeção de armazenamento em blobs, quando apresentada, permanece uma hipótese de empacotamento, sem conversão automática em economia monetária.

O custo de implantação é apresentado pelo gas total dos recibos das oito transações CREATE dos verificadores e das oito transações que implantaram as bibliotecas via CREATE2. Cada verificador é associado à biblioteca efetivamente vinculada por fonte e endereço. A soma desses dois recibos representa a implantação do par observado, sem incluir infraestrutura anteriormente disponível no fork. Há uma observação por implantação, sem estimativa de variabilidade ou amortização. A decomposição usada para as chamadas de verificação não é aplicada às implantações.

4.4 ANÁLISE ESTATÍSTICA E JUSTIFICATIVA DA REVISÃO

Seja $T_{e,d,b,c}$ a latência medida no ambiente e, dia d, bloco b e condição c. Define-se $M_{e,d,c}$ como a mediana das 100 latências medidas da condição naquela sessão. A família primária contém 16 contrastes logarítmicos diários:

$$D^{A}_{d,c} = \log\left(\frac{M_{\mathrm{WASM},d,c}}{M_{\mathrm{nativo},d,c}}\right), \tag{4.1}$$

$$D^{B}_{d,e,N} = \mathrm{mediana}_{b}\log\left(\frac{T_{e,d,b,\mathrm{u}N}}{T_{e,d,b,\mathrm{Field}_{N}}}\right), \tag{4.2}$$

$$D^{C}_{d,e} = \mathrm{mediana}_{b}\log\left(\frac{T_{e,d,b,\mathrm{compromisso}}}{T_{e,d,b,\mathrm{u64}}}\right). \tag{4.3}$$

O grupo A contém oito comparações de ambiente; B contém três larguras em cada ambiente; C contém uma comparação com compromisso por ambiente. Valores positivos representam maior latência no numerador. O custo do compromisso inclui toda a relação adicional e suas entradas públicas, não apenas o cálculo de hash isolado.

Para cada contraste j, o alvo é $\exp(\mathrm{mediana}(D_j))$ na distribuição modelada dos pares diários. A estimativa será a exponencial da mediana amostral dos logaritmos. Com 20 observa-

ções, isso equivale ao ponto médio geométrico dos dois valores centrais na escala de razões, e não à média geométrica de todos os dias.

Será testada a hipótese de mediana populacional nula para cada contraste, com alternativa bilateral, usando o teste de quantil da SciPy 1.18.0. O procedimento considera contagens estritamente abaixo e abaixo ou iguais ao valor hipotético para tratar empates; sua validade depende de observações independentes e identicamente distribuídas (18). O protocolo aplica Holm à família fixa de 16 testes, com nível de 5%. Serão apresentados intervalos pontuais de 95% e intervalos simultâneos conservadores com confiança individual de $1 - 0{,}05/16$, exponenciados para a escala de razões. Os intervalos simultâneos e os testes ajustados por Holm são procedimentos diferentes.

Para 20 observações contínuas independentes e identicamente distribuídas, os intervalos previstos usam as estatísticas de ordem 6 e 15 no caso pontual, e 4 e 17 no simultâneo. A análise não produzirá decisões para a família caso falte algum dos 20 pares elegíveis, exista contraste indefinido ou um contraste tenha todos os valores diários idênticos. A última condição é uma regra conservadora do estudo, não uma limitação universal do teste.

O teste Brunner–Munzel trabalha com duas amostras independentes (19). A abordagem inicialmente prevista no projeto foi revista porque a independência dos ciclos de uma sessão não está estabelecida e as análises anteriores apresentaram casos degenerados e limitações dos intervalos candidatos. A revisão muda também o parâmetro de interesse: o contraste diário não é a probabilidade relativa entre ciclos do desenho inicial. O teste de quantil não elimina, por si só, eventual dependência temporal entre dias.

A escolha de 20 pares é limitada por recursos. No cálculo de planejamento, assumindo distribuição contínua, independência e probabilidade de contraste positivo de 0,9, a regra conservadora baseada no menor limiar de Holm resultou em potência aproximada de 86,7%; com probabilidade de 0,8, a potência cai para 41,1%. São alternativas hipotéticas, não estimativas obtidas do piloto nem garantias de detecção de diferenças pequenas. A razão 1,20 é uma referência prática para o alvo revisado, sem teste de equivalência. A precisão desejada corresponde a uma razão entre limites superior e inferior do intervalo simultâneo de até 1,10; não haverá extensão da coleta para alcançar essa largura ou significância.

A localização dos artefatos, dos geradores das tabelas e dos registros de validação é apresentada no Apêndice A. Esse roteiro distingue a conferência de resultados retidos da execução de novos experimentos.

4.5 LIMITAÇÕES DO DESENHO

O uso de uma máquina, entradas canônicas fixas e um calendário de conveniência limita a generalização. Políticas de frequência, condições térmicas e processos concorrentes podem variar sem serem inteiramente capturados pelas observações de extremidade. A inspeção da evolução diária pode revelar padrões incompatíveis com a análise proposta, mas sua ausência não prova independência.

A interpretação final dependerá da completude da coleta e da adequação das premissas. Se a inferência não for sustentada, os resultados descritivos serão mantidos com sua limitação, sem interpretar indisponibilidade ou ausência de significância como igualdade entre variantes. O registro de incidentes e a preservação de resultados desfavoráveis integram o método, em vez de constituírem critérios para excluir observações.

5 RESULTADOS

5.1 COBERTURA E CORREÇÃO FUNCIONAL

O conjunto de validação arquivado registra a verificação local bem-sucedida das provas canônicas das oito variantes, com o alvo EVM. A suíte Noir registra 71 funções de teste aprovadas: oito para cada variante inteira, onze para cada variante Field com restrições, onze para a variante com compromisso e três para Field diagnóstico. Esses resultados correspondem aos casos de teste descritos na Seção 3.

Na validação externa de geração de testemunhas, os 46 casos inválidos exercitados foram rejeitados: três para cada variante inteira, dez para cada variante Field com restrições e sete para a variante com compromisso. Essas rejeições ocorrem na preparação da testemunha; não representam 46 provas inválidas submetidas a um verificador. O conjunto não inclui casos negativos externos para Field diagnóstico, cuja relação exclui os extremos mas não exige pertencimento ao intervalo inteiro.

A suíte Foundry registra quinze funções de teste aprovadas, abrangendo aceitação das provas válidas e rejeição de alterações de provas ou entradas públicas, entre outras verificações descritas na implementação. Algumas funções percorrem múltiplos casos; por isso, esse total não é somado aos 46 casos de testemunha como se todos representassem a mesma unidade de validação. Os resultados Foundry também não substituem os recibos de transação necessários à comparação final de gas.

Em conjunto, os registros sustentam a aceitação das entradas canônicas e a rejeição dos casos inválidos exercitados nas etapas correspondentes. O alcance dessa evidência é funcional e limitado à suíte executada: não demonstra ausência de toda sub-restrição possível nem segurança formal do backend. A análise de desempenho utiliza essas relações validadas, preservando a distinção entre Field diagnóstico e as variantes de intervalo.

5.2 CUSTOS ESTRUTURAIS E VERIFICAÇÃO EVM

A Tabela 1 apresenta as contagens estruturais e os tamanhos das provas canônicas validadas. As três variantes inteiras possuem a mesma contagem ACIR, enquanto a contagem de gates difere. Nas variantes Field com restrições, a contagem ACIR também permanece cons-

tante entre larguras. Assim, a igualdade nessa contagem intermediária não implica igualdade de estrutura no backend.

Tabela 1 – Operações e tamanho das provas das oito variantes

Variante	ACIR	Gates	Prova (bytes)
Field diagnóstico	5	40	4 928
u8	11	93	5 312
u32	11	2 791	7 232
u64	11	2 838	7 232
Field restrito 8	7	91	5 312
Field restrito 32	7	2 789	7 232
Field restrito 64	7	2 836	7 232
u64 com compromisso	37	14 348	8 000

Fonte: elaboração própria a partir dos artefatos validados do experimento (2026).
Nota: ACIR e gates são contagens de camadas distintas. Provas canônicas com alvo EVM; entradas e versões descritas na Seção 3. Field diagnóstico não verifica pertencimento ao intervalo.

As provas das variantes de 32 e 64 bits têm o mesmo tamanho no conjunto apresentado, tanto para inteiros quanto para Field com restrições. A variante com compromisso possui mais operações ACIR, mais gates e uma prova maior que a variante u64 sem compromisso. Essas observações descrevem os artefatos desta configuração; a comparação de latência depende da coleta correspondente.

A Tabela 2 apresenta os tamanhos dos modelos compilados de runtime dos verificadores. Todos ficam abaixo de 24.576 bytes, com margens de 330 a 333 bytes. A margem é uma diferença aritmética em relação ao limite EIP-170. Na rodada de gas, os oito verificadores foram implantados e seus códigos conferidos, incluindo a vinculação das bibliotecas e os valores inseridos pelo construtor.

A Tabela 3 apresenta o gas observado nas 40 provas selecionadas. As 40 chamadas de verificação retornaram verdadeiro e os 80 recibos registraram sucesso. Em cada prova, as duas transações tiveram gas total idêntico. O piso de calldata não vinculou em nenhuma dessas verificações, permitindo recuperar a execução líquida em todos os casos.

Nas cinco provas de cada variante, a execução líquida permaneceu constante, enquanto o gas total variou com a composição da calldata. Os pares Field restrito 8/u8, Field restrito 32/u32 e Field restrito 64/u64 apresentaram os mesmos valores de execução líquida. As variantes de 32 e 64 bits também coincidiram nessa métrica. Essas igualdades descrevem as provas selecionadas

Tabela 2 – Tamanho dos modelos compilados de código dos verificadores

Variante	Runtime (bytes)	Margem (bytes)
Field diagnóstico	24 243	333
u8	24 246	330
u32	24 244	332
u64	24 245	331
Field restrito 8	24 245	331
Field restrito 32	24 243	333
Field restrito 64	24 245	331
u64 com compromisso	24 245	331

Fonte: elaboração própria a partir dos artefatos validados do experimento (2026).
Nota: Margem calculada em relação a 24.576 bytes (EIP-170). Os tamanhos correspondem aos modelos compilados, incluindo posições de vinculação de bibliotecas; não substituem a conferência do código efetivamente implantado. Não são medidas de gas.

Tabela 3 – Gas de verificação das oito variantes no fork local de Sepolia

	Gas total			Execução
Variante	Mínimo	Mediana	Máximo	líquida
Field diagnóstico	1 935 842	1 935 926	1 935 926	1 838 390
u8	2 003 208	2 003 292	2 003 400	1 899 684
u32	2 340 880	2 340 904	2 341 012	2 206 708
u64	2 340 796	2 340 892	2 340 964	2 206 708
Field restrito 8	2 003 196	2 003 292	2 003 352	1 899 684
Field restrito 32	2 340 832	2 340 892	2 340 952	2 206 708
Field restrito 64	2 340 904	2 340 916	2 340 964	2 206 708
u64 com compromisso	2 479 655	2 479 739	2 479 811	2 332 639

Fonte: elaboração própria a partir dos recibos da coleta de 26/10/2026.
Nota: valores em unidades de gas. Resumo de cinco provas por variante; duas transações por prova, com custos idênticos entre repetições. A execução líquida foi constante nas cinco provas. Fork no bloco 11.999.881, Anvil 1.1.0 com Prague; solc 0.8.28, alvo Cancun e parâmetro optimizer_runs=1. Field diagnóstico possui relação distinta das variantes de intervalo.

e a implementação fixada; não decorrem de um teste de equivalência sobre uma população de provas.

A variante u64 com compromisso apresentou maior custo de execução que u64 sem compromisso. Essa diferença abrange a verificação da relação adicional e de suas entradas públicas, não o custo isolado de uma operação de hash. Field diagnóstico apresentou o menor valor observado, mas verifica um predicado distinto e, portanto, não representa uma alternativa semanticamente equivalente de prova de intervalo.

Os custos de implantação estão separados na Tabela 4. Todos os 16 recibos registraram sucesso. A soma do verificador e de sua biblioteca variou de 6.670.986 a 6.671.659 gas entre as variantes. A rodada implantou uma biblioteca para cada verificador; os valores não pressupõem compartilhamento ou reaproveitamento de uma implantação anterior.

Tabela 4 – Gas total de implantação dos verificadores e bibliotecas vinculadas

Variante	Verificador	Biblioteca	Soma
Field diagnóstico	5 294 220	1 376 766	6 670 986
u8	5 294 893	1 376 766	6 671 659
u32	5 294 536	1 376 766	6 671 302
u64	5 294 632	1 376 766	6 671 398
Field restrito 8	5 294 629	1 376 766	6 671 395
Field restrito 32	5 294 260	1 376 754	6 671 014
Field restrito 64	5 294 608	1 376 766	6 671 374
u64 com compromisso	5 294 608	1 376 766	6 671 374

Fonte: elaboração própria a partir dos recibos da rodada de 26/10/2026.
Nota: valores em unidades de gas; uma transação por verificador (CREATE) e por biblioteca (CREATE2). Soma dos dois recibos associados à variante, sem amortização ou custo de infraestrutura previamente disponível no fork. Mesma configuração da Tabela 3.

5.3 EXECUÇÃO DA COLETA E INCIDENTES

Até 11 de novembro de 2026, foram concluídos dez pares diários: de 26 de outubro a 1º de novembro e nos dias 3, 6 e 11 de novembro. Os vinte registros de sessão desses pares totalizam 18.400 provas verificadas, sendo 2.400 de aquecimento e 16.000 medidas, sem falhas de geração ou verificação registradas nessas sessões completas. Esse total descreve os pares retidos

para a análise; tentativas interrompidas permanecem arquivadas separadamente. A análise final de latência depende dos dez pares restantes e da avaliação de elegibilidade e dos pressupostos.

Em 29 de outubro, uma primeira tentativa completou apenas a sessão nativa e foi interrompida antes do WASM. Uma repetição integral foi autorizada antes de sua execução; o par completo dessa segunda tentativa foi retido, preservando o registro anterior sem combinar suas sessões. Em 3 de novembro, uma interpretação de reinício levou à reutilização das sementes do primeiro par. O autor esclareceu posteriormente a intenção de continuidade, e o par foi preservado com sua proveniência original. Essas emendas ocorreram durante a coleta e devem acompanhar a avaliação final, sem serem apresentadas como parte inalterada da alocação inicial.

Não há pares coletados em 2, 4, 5 e de 7 a 10 de novembro. Em 6 de novembro, um lançamento foi encerrado durante a pausa inicial, antes de qualquer tentativa, e preservado; a coleta posteriormente autorizada nessa data foi concluída. Em 11 de novembro, o controle de espaço em disco bloqueou a execução antes das medições; após liberação de espaço e novo preflight, o par foi executado integralmente. Ausências e bloqueios anteriores a tentativas não são contados como falhas de prova.

Na preparação da rodada de gas, duas tentativas foram interrompidas em consultas de rede. Outras duas chegaram às implantações, mas pararam na conferência do código: primeiro pela omissão de bibliotecas implantadas via CREATE2 no inventário e depois pelo tratamento incompleto dos valores imutáveis inseridos pelo construtor. Os controles foram corrigidos e os registros anteriores preservados. As 80 verificações analisadas pertencem à rodada concluída após essas correções, mantendo as provas selecionadas e o mesmo bloco e hash fixados antes das implantações. O diagnóstico separado do piso resultou em 61.000 gas, conforme esperado para o vetor descrito no método.

5.4 LATÊNCIA E CONTRASTES DIÁRIOS

Pendente de redação: Inserir as estimativas dos 16 contrastes e seus intervalos após completar e validar a alocação de vinte pares. Esta versão não apresenta inferência parcial de latência.

6 DISCUSSÃO

A discussão a seguir abrange os resultados funcionais, estruturais e de verificação EVM já concluídos. As perguntas P1 e P2 dependem dos contrastes de latência da coleta diária, ainda em andamento nesta versão. Os resultados dos pilotos não são utilizados para antecipar essas respostas.

6.1 RESPOSTAS ÀS PERGUNTAS DE PESQUISA

Os resultados permitem responder a P3 no recorte da configuração adotada: as Tabelas 1 a 4 apresentam contagens estruturais, tamanhos dos artefatos e gas de verificação e implantação das oito variantes. A Tabela 1 mostra que o efeito da representação numérica depende da camada observada. As variantes inteiras têm a mesma contagem ACIR nas três larguras, mas diferem em gates e tamanho de prova. Portanto, a contagem intermediária, isoladamente, não ordena todos os custos considerados.

A comparação semanticamente controlada entre Field restrito e inteiro torna essa distinção mais precisa. Em cada largura, a variante Field utiliza menos operações ACIR e dois gates a menos que a variante inteira. Contudo, os tamanhos de prova coincidem e, nas cinco provas selecionadas para gas, a execução líquida de verificação também coincide. A redução estrutural observada não se traduziu em redução dessa métrica EVM. Esse resultado não determina qual representação terá menor latência de geração, questão que permanece vinculada à análise diária de P2.

A inclusão do compromisso amplia a relação provada e aumenta gates, tamanho da prova e gas de execução em comparação com u64 sem compromisso. Essa combinação evidencia um custo adicional nas camadas medidas, mas não permite atribuí-lo exclusivamente ao cálculo de Pedersen: o verificador processa a prova da relação ampliada e suas entradas públicas. O ganho funcional consiste em vincular o valor usado no intervalo ao compromisso informado. A autenticidade desse valor como atributo de uma pessoa ou instituição continua fora da relação implementada.

A implantação local foi possível para os oito verificadores, com código conferido e margens de 330 a 333 bytes em relação ao limite EIP-170. Essas margens caracterizam a configuração compilada, não uma garantia de que versões futuras do gerador ou outras opções de compilação também caberão no limite. A implantação inicial do par verificador/biblioteca teve

custos próximos entre as variantes, enquanto o gas de verificação apresentou diferenças mais amplas. Essa distinção separa um custo inicial do custo por uso. O estudo não estima amortização por número de verificações nem preço monetário de uma aplicação.

6.2 COMPARAÇÃO COM TRABALHOS RELACIONADOS

A apresentação por camadas concretiza, no recorte deste experimento, a recomendação de considerar múltiplas métricas de sistemas de provas formulada por Benarroch et al. (2). A coincidência de tamanho de prova e gas de execução líquida nos pares Field restrito/inteiro, apesar de contagens estruturais diferentes, ilustra por que uma única métrica seria insuficiente para caracterizar a implementação. Não se estabelece uma regra geral de correspondência entre gates, bytes e custo do verificador a partir dessas oito relações.

O relatório de Nadolinski para a Base também considera geração de provas e verificação EVM, mas utiliza uma relação de verificação de assinaturas ECDSA P-256 e outra configuração de execução (15). A comparação pertinente está na separação dessas etapas de custo. Seus valores não constituem uma referência numérica diretamente comparável às provas de intervalo deste TCC, e a rodada local não produz um ranking entre os sistemas avaliados naquele relatório.

A distinção entre predicado e aplicação completa também aparece no relatório de credenciais de Gasser, Humbert e Elghareeb, que separa a prova de idade dos mecanismos de vínculo ao titular e de assinatura do emissor (17). Neste estudo, os testes funcionais e o compromisso sustentam o comportamento das relações implementadas, sem completar esses mecanismos. Assim, o custo da variante com compromisso deve ser interpretado como custo do protótipo descrito, não como custo total de uma solução de identidade eletrônica.

Pendente de redação: Após a análise final, discutir P1 e P2 em relação aos trabalhos browser/nativo, preservando as diferenças de circuito, hardware, versões e fronteiras temporais.

6.3 IMPLICAÇÕES E AMEAÇAS À VALIDADE

A decomposição do gas evita atribuir à execução do verificador toda diferença observada no recibo. Nesta rodada, o piso não vinculou e a execução líquida foi constante dentro de cada variante; as diferenças entre suas cinco provas vieram do componente de calldata. Reportar

apenas o total ocultaria essa distinção. Se o piso determinasse o total, a mesma subtração não identificaria univocamente a execução, conforme a regra descrita no método.

As duas transações de cada prova produziram o mesmo gas, mas repetem exatamente a mesma entrada. Elas verificam a repetibilidade daquele caso, sem ampliar o conjunto de provas coberto. As cinco posições foram fixadas antes da rodada e não escolhidas por desempenho; ainda assim, não são uma amostra probabilística do espaço de provas. As igualdades de execução observadas têm esse alcance, e nenhum teste de equivalência foi aplicado.

A fixação das versões e a conferência do código implantado delimitam qual implementação foi avaliada. As interrupções na preparação do fork e na conferência de bibliotecas e valores imutáveis mostram a importância de distinguir artefato compilado, implantação e verificação efetiva. A preservação dessas tentativas permite identificar as correções anteriores à rodada completa; seus recibos não foram misturados aos 80 analisados.

Por fim, os testes negativos exercitam casos específicos, enquanto as entradas canônicas e a única máquina restringem a variedade de uso avaliada. A viabilidade para uma aplicação depende de requisitos externos de latência, recursos, custo e autenticação que não foram definidos neste experimento. A avaliação de latência ainda deverá considerar a completude dos 20 pares, as diferentes regiões cronometradas e as premissas do método; nenhum resultado de gas resolve essas condições.

7 CONCLUSÃO

7.1 SÍNTESE DAS RESPOSTAS E CONTRIBUIÇÕES

Pendente de redação: Desenvolver esta seção a partir das evidências previstas no plano de produção.

7.2 LIMITAÇÕES E TRABALHOS FUTUROS

Pendente de redação: Desenvolver esta seção a partir das evidências previstas no plano de produção.

REFERÊNCIAS

1 ZKPROOF. **ZKProof Community Reference**. Versão 0.3, 17 jul. 2022. Documento em desenvolvimento. Disponível em: https://docs.zkproof.org/reference.pdf. Acesso em: 24 out. 2026.

2 BENARROCH, Daniel; NICOLAS, Aurelien; THALER, Justin; TROMER, Eran. **Community Proposal: A Benchmarking Framework for (Zero-Knowledge) Proof Systems**. 9 abr. 2020. Disponível em: https://docs.zkproof.org/pages/standards/accepted-workshop3/proposal-benchmarking.pdf. Acesso em: 24 out. 2026.

3 BUTERIN, Vitalik. **EIP-170: Contract code size limit**. 2016. Disponível em: https://eips.ethereum.org/EIPS/eip-170. Acesso em: 24 out. 2026.

4 BÜNZ, Benedikt et al. **Bulletproofs: Short Proofs for Confidential Transactions and More**. 2018. Versão disponibilizada pelos autores. Disponível em: https://web.stanford.edu/~buenz/pubs/bulletproofs.pdf. Acesso em: 24 out. 2026.

5 NOIR. **Noir Documentation**. Página inicial, versão 1.0.0-rc.5. Versão documental registrada em 15 out. 2026 no repositório oficial. Disponível em: https://noir-lang.org/docs/. Acesso em: 27 out. 2026.

6 AZTEC PROTOCOL. **Ultra Honk**. Documentação do Barretenberg, versão 5.0.0-nightly.20260324, 2026. Disponível em: https://github.com/AztecProtocol/aztec-packages/blob/7ac4a3c97c4c0b9056edac36f207b7ef839877fc/barretenberg/cpp/src/barretenberg/ultra_honk/README.md. Acesso em: 25 out. 2026.

7 WEBASSEMBLY COMMUNITY GROUP. **WebAssembly Core Specification: Introduction**. Versão 3.0, 23 out. 2026. Disponível em: https://webassembly.github.io/spec/core/intro/introduction.html. Acesso em: 24 out. 2026.

8 WAHRSTÄTTER, Toni; BUTERIN, Vitalik. **EIP-7623: Increase calldata cost**. 2024. Disponível em: https://eips.ethereum.org/EIPS/eip-7623. Acesso em: 24 out. 2026.

9 BUTERIN, Vitalik et al. **EIP-4844: Shard Blob Transactions**. 2022. Disponível em: https://eips.ethereum.org/EIPS/eip-4844. Acesso em: 24 out. 2026.

10 KALIBERA, Tomas; JONES, Richard. **Rigorous Benchmarking in Reasonable Time**. ISMM, 2013. p. 63–74. DOI: 10.1145/2464157.2464160. Versão aceita dos autores. Disponível em: https://kar.kent.ac.uk/33611/. Acesso em: 24 out. 2026.

11 ERNSTBERGER, Jens et al. **zk-Bench: A Toolset for Comparative Evaluation and Performance Benchmarking of SNARKs**. Cryptology ePrint Archive, Paper 2023/1503, 2023. Preprint. Disponível em: https://eprint.iacr.org/2023/1503. Acesso em: 25 out. 2026.

12 JANGDA, Abhinav; POWERS, Bobby; BERGER, Emery; GUHA, Arjun. **Not So Fast: Analyzing the Performance of WebAssembly vs. Native Code**. arXiv:1901.09056, versão 3, 2019. Disponível em: https://arxiv.org/abs/1901.09056v3. Acesso em: 25 out. 2026.

13 MACH34. **ZKEmail.nr Benchmarks**. Repositório de código, revisão f11c1d20ac5fda512bec8260914e9b14c7309231. README revisado em 5 fev. 2025. Disponível em: https://github.com/Mach-34/zkemail.nr-bench/tree/f11c1d20ac5fda512bec8260914e9b14c7309231. Acesso em: 27 out. 2026.

14 DUFF, Joss. **Privacy-Preserving Compliance**. Dissertação (Master of Science in Computer Science) — Lehigh University, 2026. Fonte LaTeX disponibilizada pelo autor. Disponível em: https://github.com/JossDuff/thesis/blob/12226bfcf151b66b68d8e3b25fd0a9d98bf08b34/main.tex. Acesso em: 25 out. 2026.

15 NADOLINSKI, Elena. **Benchmarking ZKP Systems for Passkey ECDSA Verification**. Base, 2 set. 2025. Relatório técnico publicado no blog de engenharia. Disponível em: https://www.base.org/blog/benchmarking-zkp-systems. Acesso em: 25 out. 2026.

16 WORLD FOUNDATION. **Reproducing the V1 input-to-proof campaign**. Documentação do repositório ProveKit, revisão b8c7dccff0e2af31095fc9e88e18156c515cce46. Documento revisado em 14 ago. 2026. Disponível em: https://github.com/worldfnd/provekit/blob/b8c7dccff0e2af31095fc9e88e18156c515cce46/benchmarks/v1/REPRODUCIBILITY.md. Acesso em: 27 out. 2026.

17 GASSER, Linus; HUMBERT, Clement; ELGHAREEB, Ahmed. **Secure and Privacy-Preserving Credentials for E-ID: Proof-of-Concept**. Relatório técnico, fonte Typst. Versão distribuída em 22 out. 2026, release 2026-10-22_16-34, correspondente ao commit fc07979aba93be43897bcaeb813b971b15ad9226. Disponível em: https://github.com/eid-privacy/MS2-Minimal-Viable-Project/blob/fc07979aba93be43897bcaeb813b971b15ad9226/report/index.typ. Acesso em: 27 out. 2026.

18 SCIPY. **quantile_test — SciPy v1.18.0 Manual**. Edição documental de 19 jun. 2026. Disponível em: https://docs.scipy.org/doc/scipy-1.18.0/reference/generated/scipy.stats.quantile_test.html. Acesso em: 27 out. 2026.

19 SCIPY. **brunnermunzel — SciPy v1.18.0 Manual**. Edição documental de 19 jun. 2026. Disponível em: `https://docs.scipy.org/doc/scipy-1.18.0/reference/generated/scipy.stats.brunnermunzel.html`. Acesso em: 27 out. 2026.

APÊNDICE A – RASTREABILIDADE DOS ARTEFATOS E DOS RESULTADOS

Os caminhos deste apêndice são relativos à raiz do repositório de trabalho. Eles identificam os materiais usados na elaboração das tabelas e na verificação dos resultados. A organização descrita não implica que o conjunto completo esteja publicado ou depositado em um serviço externo. Os registros de execução devem ser distinguidos dos arquivos produzidos para apresentação no manuscrito.

Validação funcional e identificação das variantes. O diretório `results/spec008/final-correctness/` reúne o manifesto, fontes dos circuitos, provas canônicas, entradas públicas e chaves de verificação por variante. Os arquivos `manifest.json`, `foundry-tests.json` e `final-review.json`, nesse diretório, documentam a validação arquivada. O vetor do compromisso está em `commitment-vector.json`. A conferência das contagens utilizadas no texto está em `manuscript/validation/functional-evidence.json`. Esses registros permitem localizar os casos executados; sua consulta não constitui uma nova execução dos testes.

Métricas estruturais. O arquivo `results/final-readiness-20261024/coverage.csv` fornece as métricas confrontadas com o manifesto de validação. O gerador `manuscript/scripts/build_structural_tables.py` produz as tabelas estruturais e registra a proveniência dos arquivos utilizados. A identidade dos artefatos deve ser conferida antes de interpretar uma diferença entre valores regenerados e os apresentados no texto.

Verificação e implantação EVM. A rodada usada nas tabelas de gas está em `results/final-gas-20261026-attempt05/`. O arquivo `fork-block.json` identifica o bloco do fork; `records.json` relaciona as provas e transações; `summary.json` contém os resumos por variante. Os recibos individuais, a calldata e os registros das implantações permanecem nesse conjunto. O inventário `artifact-hashes.json` registra os resumos criptográficos dos artefatos. As tentativas anteriores são preservadas como histórico dos incidentes e não integram as tabelas finais.

O script `manuscript/scripts/build_gas_table.py` reconstrói os valores de verificação a partir dos recibos e da calldata. O script `manuscript/scripts/build_deployment_table.py` confere os vínculos entre verificadores e bibliotecas e produz a tabela de implantação. Ambos escrevem arquivos de apresentação e de proveniência no diretório do manuscrito; não enviam transações. Regenerar essas tabelas verifica a transformação dos

dados retidos em valores apresentados, mas não repete a execução EVM nem constitui validação por uma implementação independente.

Protocolo e registros diários. O calendário e o manifesto de arquivos congelados estão em `specs/016-final-analysis/schedule.json` e `specs/016-final-analysis/frozen-manifest.json`. Os registros da coleta final são organizados por data dentro de `results/final-daily-20261025/` e `results/final-daily-20261103/`. O sufixo dessa raiz é um identificador histórico: a data efetiva deve ser consultada no subdiretório e nos registros da execução. O estado consolidado está em `docs/daily-collection-status.md`; o procedimento operacional está em `docs/daily-collection-runbook.md`. Os pilotos são conjuntos distintos e não completam dias ausentes da amostra final.

A análise prevista está implementada em `analysis/final_daily.py`. Sua presença no repositório não demonstra que a coleta esteja completa ou que a análise primária seja elegível. A interpretação depende das condições de completude e validade descritas no método, preservando falhas e indisponibilidades.

Fontes bibliográficas e apresentação. O arquivo `manuscript/sources/verification.md` registra o alcance das consultas bibliográficas; os inventários associados identificam os documentos arquivados. O mapa `manuscript/evidence-map.md` relaciona seções e evidências. O script `manuscript/build.sh` compila a monografia a partir dos arquivos LaTeX e das tabelas existentes. Compilar o documento não executa os coletores nem atualiza automaticamente os resultados. O diário `docs/research-log.md` preserva as decisões e correções necessárias para interpretar o histórico.